

■

CONTEST PROBLEM-SOLVER

CHEAT SHEET

Competitive Programming · Quick Reference · All Topics
Algorithms · Data Structures · Math · Graph Theory · DP · Strings

■

TIME COMPLEXITY — DECIDE YOUR APPROACH

N ≤	Viable Complexity	Typical Approach
10	$O(N!)$	Brute force / permutations
20	$O(2^N)$	Bitmask DP / subset enum
100	$O(N^3)$	Floyd-Warshall / naive DP
1 000	$O(N^2)$	All-pairs, bubble sort
10 000	$O(N^2 / \log)$	Two-pointer, careful DP
100 000	$O(N \log N)$	Sort, segment tree, BFS/DFS
1 000 000	$O(N) / O(N \log N)$	Greedy, hash map, sliding window
10^8	$O(N)$	Single pass only
10^{18}	$O(\log N)$	Binary search, fast exponentiation

■

MEMORY & TYPE LIMITS

Type	Size	Max Value	Use For
int	4 B	$\sim 2.1 \times 10^9$	Default integers
long long	8 B	$\sim 9.2 \times 10^{18}$	Large counts / products
double	8 B	$\sim 1.8 \times 10^3$ (15 dp)	Geometry, probability
__int128	16 B	$\sim 1.7 \times 10^3$	Huge intermediate products
bool[]	1 B	true/false	DP tables, visited
bitset<N>	N/8 B	bit flags	DP optimisation, SOS

■

FAST I/O (C++)

```
ios_base::sync_with_stdio(false);
cin.tie(NULL);
// For huge input use scanf/printf or custom reader
#define int long long // avoid int overflow bugs
const int MOD = 1e9+7;
const int INF = 1e18;
```

■

STL QUICK REFERENCE

Container	Key Operations	Time
vector<T>	push_back, pop_back, [i]	$O(1)$ amort.
deque<T>	push/pop front & back	$O(1)$

■ SORTING & SEARCHING

Algorithm	Best	Average	Worst	Space	Stable
<code>std::sort</code>	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(\log N)$	No
<code>std::stable_sort</code>	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$	Yes
Counting Sort	$O(N+K)$	$O(N+K)$	$O(N+K)$	$O(K)$	Yes
Radix Sort	$O(dN)$	$O(dN)$	$O(dN)$	$O(N+K)$	Yes
Binary Search	$O(1)$	$O(\log N)$	$O(\log N)$	$O(1)$	—

Binary Search Template

```
int lo=0, hi=n-1, ans=-1;
while(lo<=hi){
    int mid=lo+(hi-lo)/2;
    if(check(mid)){ ans=mid; lo=mid+1; } // last true
    else hi=mid-1;
}
```

■ GRAPH ALGORITHMS

Algorithm	Time	Space	Use Case
BFS	$O(V+E)$	$O(V)$	Shortest path (unweighted)
DFS	$O(V+E)$	$O(V)$	Connectivity, topo sort
Dijkstra (PQ)	$O((V+E) \log V)$	$O(V)$	SSSP non-negative weights
Bellman-Ford	$O(VE)$	$O(V)$	SSSP with negative edges
Floyd-Warshall	$O(V^3)$	$O(V^2)$	APSP
Prim / Kruskal	$O(E \log E)$	$O(V)$	MST
Topological Sort	$O(V+E)$	$O(V)$	DAG ordering
Tarjan SCC	$O(V+E)$	$O(V)$	Strongly connected comp.
Articulation Points	$O(V+E)$	$O(V)$	Bridge / cut vertices
Bipartite Check	$O(V+E)$	$O(V)$	2-coloring via BFS

Dijkstra Skeleton

```
priority_queue<pii,vector<pii>,greater<pii>> pq;
dist[src]=0; pq.push({0,src});
while(!pq.empty()){
    auto[d,u]=pq.top(); pq.pop();
    if(d>dist[u]) continue;
    for(auto[v,w]:adj[u])
        if(dist[u]+w<dist[v]){ dist[v]=dist[u]+w; pq.push({dist[v],v}); }
}
```

■ UNION-FIND (DSU)

```
int par[N],rnk[N];
void init(int n){iota(par,par+n,0); fill(rnk,rnk+n,0);}
int find(int x){return par[x]==x?x:par[x]=find(par[x]);}
bool unite(int a,int b){
    a=find(a);b=find(b);if(a==b)return false;
    if(rnk[a]<rnk[b])swap(a,b);
    par[b]=a;if(rnk[a]==rnk[b])rnk[a]++;return true;
}
```

■ DYNAMIC PROGRAMMING

Pattern	State	Transition	Complexity
0/1 Knapsack	$dp[i][w]$	$\max(dp[i-1][w], dp[i-1][w-w_i]+v_i)$	$O(NW)$
Unbounded Knapsack	$dp[w]$	$dp[w] = \max(dp[w], dp[w-w_i]+v_i)$	$O(NW)$
LCS	$dp[i][j]$	$dp[i-1][j-1]+1$ or $\max(dp[i-1][j], dp[i][j-1])$	$O(NM)$
LIS (patience)	$tails[]$	lower_bound + replace	$O(N \log N)$
Edit Distance	$dp[i][j]$	$\min(\text{ins}, \text{del}, \text{sub})+1$	$O(NM)$
Coin Change (min)	$dp[v]$	$\min(dp[v], dp[v-c_i]+1)$	$O(NV)$
Interval DP	$dp[l][r]$	min/max over split k	$O(N^3)$
Bitmask DP	$dp[\text{mask}]$	$dp[\text{mask} (1 \ll i)]$	$O(N \cdot 2^N)$
Digit DP	$dp[\text{pos}][\text{tight}][\dots]$	recurse on digits	$O(\log N \cdot \text{states})$
Tree DP	$dp[v][\text{state}]$	combine children subtrees	$O(N \cdot \text{states})$

LIS — $O(N \log N)$

```
vector<int> tails;
for(int x:a){
    auto it=lower_bound(tails.begin(),tails.end(),x);
    if(it==tails.end()) tails.push_back(x);
    else *it=x;
}
// tails.size() = length of LIS
```

Knapsack 0/1 — Space $O(W)$

```
for(int i=0;i<n;i++){
    for(int w=W;w>=w_i[i];w--) // reverse!
        dp[w]=max(dp[w], dp[w-w_i[i]]+v_i[i]);
}
```

■ MATH ESSENTIALS

Topic	Formula / Code
GCD / LCM	<code>__gcd(a,b)</code> $a/\gcd(a,b) \cdot b$
Fast Power	<code>power(a,b,mod): (b&1?a:1)*power(a*a%m,b>>1,m)%m</code>
Modular Inverse	<code>power(a, MOD-2, MOD)</code> (MOD prime)
$nCr \bmod p$	<code>fact[n]*inv_fact[r]%p*inv_fact[n-r]%p</code>
Sieve of Eratost.	$O(N \log \log N)$ – mark composites
Euler's Totient	$\phi(n) = n \prod (1-1/p)$ for prime $p n$
Matrix Exp	Solve linear recurrence in $O(k^3 \log n)$
Chinese Rem. Thm.	$x \equiv a_i \pmod{n_i} \dots$ combine pairwise
Inclusion-Exclusion	$ A \cup B \cup C = \sum A_i - \sum A_i \cap A_j + \dots$
Catalan Numbers	$C(n)=C(2n,n)/(n+1)$: balanced parens, BSTs

■ DATA STRUCTURES

Segment Tree (range sum + point update)

```
int tree[4*N];
void build(int*a,int node,int s,int e){
    if(s==e){tree[node]=a[s];return;}
    int m=(s+e)/2;
    build(a,2*node,s,m); build(a,2*node+1,m+1,e);
    tree[node]=tree[2*node]+tree[2*node+1];
}
void update(int node,int s,int e,int i,int v){
    if(s==e){tree[node]=v;return;}
    int m=(s+e)/2;
    if(i<=m)update(2*node,s,m,i,v);
    else update(2*node+1,m+1,e,i,v);
    tree[node]=tree[2*node]+tree[2*node+1];
}
int query(int node,int s,int e,int l,int r){
    if(r<s||e<l)return 0;
    if(l<=s&&e<=r)return tree[node];
    int m=(s+e)/2;
    return query(2*node,s,m,l,r)+query(2*node+1,m+1,e,l,r);
}
```

BIT / Fenwick Tree (prefix sum)

```
int bit[N];
void upd(int i,int v){for(;i<N;i+=i&-i)bit[i]+=v;}
int qry(int i){int s=0;for(;i>0;i-=i&-i)s+=bit[i];return s;}
// range [l,r]: qry(r)-qry(l-1)
```

Sparse Table (static RMQ, O(1) query)

```
int sp[N][LOG]; int lg[N];
void build(){
    lg[1]=0; for(int i=2;i<N;i++)lg[i]=lg[i/2]+1;
    for(int i=0;i<n;i++)sp[i][0]=a[i];
    for(int j=1;(1<<j)<=n;j++)
        for(int i=0;i+(1<<j)-1<n;i++)
            sp[i][j]=min(sp[i][j-1],sp[i+(1<<(j-1))][j-1]);
}
int rmq(int l,int r){int k=lg[r-l+1];
return min(sp[l][k],sp[r-(1<<k)+1][k]);}
```

Trie (prefix tree)

```
struct Trie{
    int ch[N][26],cnt[N],tot=1;
    void insert(string&s){
        int cur=0;
        for(char c:s){int i=c-'a';
            if(!ch[cur][i])ch[cur][i]=tot++;
            cur=ch[cur][i];}
        cnt[cur]++;}
};
```

Structure	Query	Update	Space	Best For
Array	$O(N)$	$O(1)$	$O(N)$	Sequential access
Fenwick Tree	$O(\log N)$	$O(\log N)$	$O(N)$	Prefix sums

■ STRING ALGORITHMS

Algorithm	Time	Use Case
KMP	$O(N+M)$	Pattern matching
Z-Function	$O(N)$	Pattern matching, period
Rabin-Karp	$O(N+M)$ avg	Multi-pattern hash match
Suffix Array	$O(N \log N)$	All substrings, LCP
Suffix Automaton	$O(N)$	Distinct substrings, occ. count
Aho-Corasick	$O(N+M+out)$	Multiple pattern search
Manacher	$O(N)$	Longest palindromic substring
Hashing (poly)	$O(N) / O(1)Q$	Rolling hash comparisons

Polynomial Rolling Hash

```
const ll BASE=131, MOD=1e9+7;
vector<ll> h(n+1,0),pw(n+1,1);
for(int i=0;i<n;i++){
h[i+1]=(h[i]*BASE+s[i])%MOD;
pw[i+1]=pw[i]*BASE%MOD;
}
// hash(l,r) = (h[r+1]-h[l]*pw[r-l+1]%MOD+MOD)%MOD
```

■ GEOMETRY BASICS

```
typedef complex<double> P;
#define x real() #define y imag()
double cross(P a,P b){return (conj(a)*b).y;}
double dot (P a,P b){return (conj(a)*b).x;}
// Polygon area (shoelace):
double area(vector<P>&p){double s=0;int n=p.size();
for(int i=0;i<n;i++)s+=cross(p[i],p[(i+1)%n]);
return abs(s)/2;}
// Convex Hull – Andrew's monotone chain:  $O(N \log N)$ 
```

Operation	Formula/Method
Point in polygon	Ray casting or winding number
Line intersection	Parametric: $P + t \cdot D = Q + s \cdot E$, solve t, s
Circle-line distance	$ cross(d, P-A) / d $
Convex Hull	Andrew's monotone chain $O(N \log N)$
Rotating Calipers	Diameter of convex hull $O(N)$
Pick's Theorem	$A = I + B/2 - 1$ (lattice polygons)

■ GREEDY PATTERNS

Problem Type	Key Insight
Interval Scheduling	Sort by end time; pick non-overlapping
Activity Selection	Same as above – Greedy = optimal
Huffman Coding	Min-heap; merge two smallest repeatedly
Fractional Knapsack	Sort by value/weight descending
Coin Change (min coins)	Only works with canonical coin sets
Gas Station	Track cumulative surplus; find start

■ CONTEST DECISION CHECKLIST

■ READ

- Read the problem TWICE — note constraints carefully
- Identify: optimise / count / construct / decision?
- Check: N, Q, values range, time limit, memory limit

■ APPROACH

- Map N to complexity → pick algorithm family
- Brute force first if $N \leq 10^4$ — optimise later
- Think about DP when: overlapping subproblems, optimal substructure
- Think about Greedy when: exchange argument holds
- Think about Graph when: nodes, edges, connectivity, paths

■ IMPLEMENT

- Use long long by default (#define int long long)
- Add MOD whenever products/sums can exceed 10^9
- Handle edge cases: $n=0$, $n=1$, empty arrays, $k>n$
- Off-by-one: are indices 0-based or 1-based?
- Overflow: $(a+b)$ may overflow before $\%MOD$ — use $(a\%m+b\%m)\%m$

■ DEBUG

- Test with sample, then small hand-crafted edge cases
- Stress test against brute force for ~500 random inputs
- Check: TLE → reduce complexity; MLE → reduce arrays; WA → logic

■ NUMBER THEORY SNIPPETS

```
// Sieve of Eratosthenes
vector sieve(int n){
    vector is_prime(n+1,true);
    is_prime[0]=is_prime[1]=false;
    for(int i=2;i*i<=n;i++)
        if(is_prime[i])
            for(int j=i*i;j<=n;j+=i)is_prime[j]=false;
    return is_prime;
}

// Fast modular exponentiation
ll power(ll a,ll b,ll m){
    ll res=1; a%=m;
    while(b>0){if(b&1)res=res*a%m; a=a*a%m; b>>=1;}
    return res;
}

// Extended GCD (for modular inverse when MOD not prime)
ll extgcd(ll a,ll b,ll&x,ll&y){
    if(!b){x=1;y=0;return a;}
    ll x1,y1,g=extgcd(b,a%b,x1,y1);
    x=y1; y=x1-(a/b)*y1; return g;
}
```

■ COMMON PITFALLS

- Integer overflow: use ll; multiply before modding
- Array out-of-bounds: size+1 for 1-indexed; check BFS/DFS bounds
- Floating point: use $\text{eps}=1e-9$; avoid $==$ comparison; prefer long double
- Stack overflow: DFS on $N=10^5+$ → convert to iterative or increase stack
- Wrong MOD: $(a-b+MOD)\%MOD$ for subtraction; mod after every multiply

- *Uninitialized: `memset(dp, 0x3f, sizeof dp)` for INF; 0 for min-cost*
 - *Re-read constraints: 'distinct', 'non-negative', 'connected' matter!*
 - *Wrong graph: directed vs undirected; weighted vs unweighted*
-

Good luck & trust your preparation! ■